Machine Learning for Tree Structures in Fake Site Detection

Taichi Ishikawa
t.i.tkgw@gmail.com
Carnegie Mellon University, USA

Yu-Lu Liu
yulu.liu@rakuten.com
Rakuten, Inc, Japan

David Lawrence Shepard
shepard.david@gmail.com
Evidation Health, USA

Kilho Shin
yoshihiro.shin@gakushuin.ac.jp
Gakushuin University, Japan

ABSTRACT

Tree data analysis has many applications in information security. In particular, HTML pages' DOM trees are an important target of analysis because web pages can be vectors for, and targets of, major cyberattacks like phishing. Previous attempts to incorporate tree data analysis into security applications, however, have been hampered by the lack of efficient methods for tree data analysis in machine learning. As such, most security research has focused on data representable as vectors of real numbers, like most machine learning work. Recent work, however, has yielded several efficiency breakthroughs in tree analysis. One example is kernel methods, a methodological bridge that fills the gap between discretely-structured data (like trees) and multivariate analysis. Kernel methods enable applying a variety of multivariate analysis techniques such as SVM and PCA to trees. The method we are interested in is the subpath kernel. The subpath kernel offers the following advantages: (1) it is invariant over ordered and unordered trees; (2) it can be computed using an extremely fast linear-time algorithm compared to the quadratic time required to compute values of most tree kernels; (3) its excellent prediction accuracy has been proven through intensive experiments. This paper proposes a subpath kernel-based method for tree-structured security data. To demonstrate the effectiveness of our method, we apply it to the problem of detecting fake e-commerce sites, a sub-problem of phishing detection with a significant real-world financial cost. In an experiment on a real dataset of fake sites provided by a major e-commerce company, our method exhibited accuracy as high as 0.998 when training SVM with as few as 1,000 instances. Its generalization efficiency is also excellent: with only 100 training instances, the accuracy score reaches 0.996. While previous phishing detection methods relied on textual content, URL components, and blacklists, our approach is the first to leverage DOM trees, which makes it both more effective and more robust against adversarial attacks. Unlike URL or content changes, changing a page's DOM structure incurs large costs to criminals.

ARES 2020, August 25–28, 2020, Virtual Event, Ireland

ACM ISBN 978-1-4503-8833-7/20/08…$15.00
https://doi.org/10.1145/3407023.3407035

CCS CONCEPTS

• **Computer systems organization** → **Security and privacy**; *Software and application security*; • **Networks** → Web application security.

KEYWORDS

fake sites detection, kernel method, web security

ACM Reference Format:
Taichi Ishikawa, Yu-Lu Liu, David Lawrence Shepard, and Kilho Shin. 2020. Machine Learning for Tree Structures in Fake Site Detection. In *The 15th International Conference on Availability, Reliability and Security (ARES 2020), August 25–28, 2020, Virtual Event, Ireland*. ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3407023.3407035

1 INTRODUCTION

Trees are a fundamental data structure in computer science, and in information security as well. Many real-world applications have proven successful by leveraging rich structural information born in trees. For example, in natural language processing, converting a sentence, a sequence of words, into what is known as a parse tree, enables the computer to approximate the meaning of the sentence. Information security is full of such examples. However, few machine learning methods are available for working with tree data. While some approaches such as tree edit distances do exist, the majority of machine learning methods, both in general research and in information security applications, work with data represented as vectors of real numbers.

Machine learning has many applications in information security, such as in intrusion detection systems (IDS). Present-day machine learning research seems dominated by deep learning, and security machine learning research is no exception. As many advances as deep learning has offered, though, there are many areas in which it does not work well. Deep learning has two major disadvantages: first, it usually requires a large quantity of training data, because artificial neural network systems include large numbers of parameters. Often, collecting sufficiently large number of data is impossible. Second, it often works well only on sequential data, or images. Many domains use data that cannot be represented as vectors, matrices or sequences. Information security, in particular, often deals extensively with trees: two examples are users' web navigation histories and the DOM structure of HTML pages.

Another advantage of this approach lies in the fact that many techniques can be extended to apply to types of data other than vector data. Schönberg's theorem [2] (see also Theorem 1 in Section 3.1) makes this possible under the condition that a particular

function, referred to as a kernel, is given. For example, the use of kernels has allowed the study of machine learning techniques on tree-type data to make remarkable progress. One approach that has yielded results has been in the area of tree kernels: A tree kernel provides a means to view tree-type data as points in Euclidean spaces, and consequently, it makes possible to apply many machine learning techniques developed based on multivariate analysis to tree-type data. Built on the fundamental results of convolution kernels[9] and mapping kernels[23], many tree kernels have been presented and investigated in the literature. The SubPath Kernel (SPK) is one example. SPK is of the few methods known in the literature to have excellent accuracy. Furthermore, SPK has been shown to perform in linear time proportional to the size of inputted trees[12, 22], which makes it feasible to apply in the time-sensitive domain of information security.

This paper presents an SPK-based method for web-page analysis. This method allows us to study the tree-base structures of web pages in an efficient manner, a problem that leading-edge research in web security has thus far been unable to address. To show our method's effectiveness and efficiency, we apply it to the problem of fake e-commerce site detection.

2 AN EXAMPLE PROBLEM – FAKE E-COMMERCE SITE DETECTION

Fake e-commerce sites have been a serious threat to both consumers and providers of e-commerce. They adopt the look of legitimate companies and attempt to obtain money by fraud, steal personal information, and ruin the reputations of platform operators. The number of reported incidents of fake e-commerce sites started to increase in 2012, and continues to rise even to the present day. In Japan, the monetary damage from phishing grew from 48 million JPY in 2012 to 2.9 billion JPY in 2014 and 3.1 billion JPY in 2015. Although the issue of fake e-commerce sites is a sub-problem of the widely known problem of phishing sites, we focus on this issue, because we were able to obtain a real dataset from one of the largest e-commerce platform operators in Japan.

2.1 The Dataset

Rakuten is a Japanese electronic commerce and Internet company, which operates *Rakuten Ichiba*, the largest e-commerce platform in Japan.

Rakuten has suffered significant monetary damages from fake sites, and is seriously concerned about its trust in the market. As a countermeasure, Rakuten has been collecting reports of fake sites from customers through help desks. Simultaneously, Rakuten operates its own *Internet Patrol Team*, which collects a range of suspicious URLs through web crawlers. The team then manually investigates whether the collected sites are fake or authentic. Once the team has concluded that a site is fake, Rakuten reports them to law enforcement agencies and Internet providers to prevent consumers from accidentally visiting such sites. As a byproduct of their work, Rakuten has accumulated a number of URLs of fake and authentic sites.

Rakuten provided us a sample of this dataset. It consists of *fresh* URLs of 3,597 fake and 3,349 authentic sites, which were live when we started this research. This is very important, because, as [17] reported, accurate annotation of datasets is important for phishing site detection. After receiving the data, we downloaded the HTML file of each page and then extracted their DOM trees for analysis.

The number of vertices of DOM trees derived from each HTML file varies from 6 to 6,565, and the mean number of nodes on each of the 6,948 DOM trees is 1010.5.

2.2 Problem Setting

We subdivided the Rakuten dataset into training and test sets. The training dataset consists of 500 fake sites and 500 authentic sites selected at random, while the test dataset contains the remainder (3,097 fake sites and 2,849 authentic sites). We used the training dataset to train a model and then tested the model on the test set. We measured the accuracy of the model's predictions to evaluate our method. Our method generalizes well and generates accurate models with a relatively small amount of training data. Because of this property, we can leverage the holdout method instead of cross-validation, where we allocate relatively older instances to training and newer instances to testing. We used these datasets in the preliminary experiments described in Section 3.1 and the experiments that demonstrate the performance of our method in Section 4.

2.3 Related Work: Phishing Site Detection

Many good surveys of phishing site detection methods are available in the literature. (*e.g.* [1, 14, 17, 29]). The majority of approaches discussed use page content, site URLs, and balacklists.

- Page content can provide finely-grained information, and the range of features encompasses hyperlinked URLs [5], linked images [5, 15], embedded executable code [20] as well as statistics such as terms used [19], tag contents [5], content types [19] and term TF-IDF values [26, 30]. On the other hand, the detailed investigation of page content may damage users' computers by executing malware on the sites, and features extracted from texts tend to be language-specific. Additionally, page content is easy to alter.
- The URL of a website is also known to provide useful clues for phishing detection. In addition to tokens that constitute the URL [18, 26, 30], metadata obtained from external sources such as DNS [26], Google Pagerank [26] and Amazon Alexa [19] are also useful. However, criminals can use link redirection and change page content to circumvent detection.
- A blacklist is created in a concentrative manner and specifies URLs of known phishing sites. Since its trustworthiness is high, it is widely used in real services like Google Safe Browsing [8] and eBay Toolbar [29]. Nevertheless, automated crawlers can be easily detected, and also, changing page content can easily render blacklists obsolete. The most significant disadvantage of blacklists is the time difference between discovery of phishing sites and their registration in blacklists.

2.4 Our Approach

Our novel approach leverages totally a new source of information, the DOM structure of web pages. A DOM tree of a web page is the

tree that represents the nested structure of the page's HTML tags. Fig. 1 shows an imaginary example, but it is common for real DOM trees to include thousands of vertices.

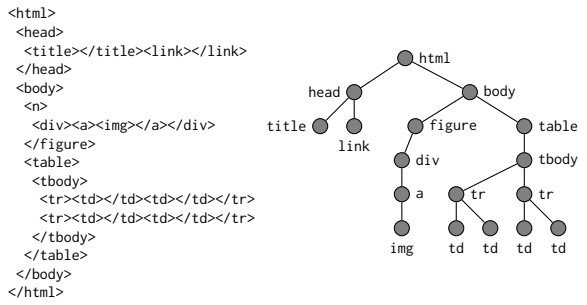

Figure 1: An HTML document and the associated DOM tree.

Our approach is not only novel and leverages the latest results of machine learning research, but is expected to be effective for the reasons stated below. Law enforcement agencies and e-commerce companies have been cooperating to identify and analyze fake sites. Once they discover fake sites, they add the detected URLs to public blacklists and request major Internet providers to block consumers' accesses to the URLs. They have been successful in shutting down thousands of fake sites. Through our joint research with Rakuten and Hyogo Prefectural Police, we have found that the average lifetime of a fake site is short, and criminals must continuously pay the cost of developing and releasing new fake sites. Also, a great deal of evidence suggests that a single group of criminals must operate a large number of fake sites simultaneously to make enough money for phishing to be worthwhile. Thus, the time and effort that criminals can allocate to developing an individual fake site is limited, and the option to build a new site from the scratch every time is not acceptable. On the other hand, they have to make individual sites look different to fool consumers. Considering the cost-performance ratio, changing content and layout is an affordable option. By contrast, modifying page structure significantly is expensive. Thus, we expect that DOM-trees of webpages will remain similar enough to discriminate fake sites from authentic ones.

3 DOM TREE ANALYSIS BY SUBPATH KERNEL

The first part of this section provides a brief review of prior machine learning techniques for tree-structured data. Then, we give a more detailed description of methods that apply to tree kernels.

3.1 Similarity Measures for Trees

Machine learning is often understood as a science of data similarity. When focusing on our problem, the key for success is evidently how effectively and efficiently we can detect the similarity between the DOM trees of sites that have the same class (fake or authentic) and the dissimilarity between sites in the opposite classes.

In fact, the similarity of tree-structured data has attracted researchers' attention for a long time. In 1979, Taï [25] extended the well-known Levenshtein edit distance [13] to trees, providing for the first time a similarity measure for trees. A Levenshtein distance quantifies the similarity of two strings by counting the minimum number of insertion, deletion and substitution operations to convert one string into the other. In the same way, when we compute the Taï distance between two trees, we perform three types of edit operations: insertion of a single vertex to a tree, deletion of a single vertex from a tree, and substitution of a new vertex for a vertex of a tree. With only these three operations, we are able to convert one tree to the other, and the Taï distance is determined by the minimum number of operations necessary. Many papers have reported the effectiveness of the Taï distance when used with distance-based classifiers such as the k nearest neighbor classifier. In fact, many variation of the Taï distance [25] including the constrained edit distance [27] and the degree-two edit distance [16, 28] have been invented and widely used in practice.

Much like an edit distance, a tree kernel is a bivariate function to evaluate the similarity of two trees specified as arguments of the function. Unlike the tree edit distance, however, a greater value of a kernel function implies greater similarity. The first instance of tree kernels was introduced by Collins and Duffy [4] in 2001, which is referred to as the *parse tree kernel*.

The most important property of the parse tree kernel is positive definiteness.

Definition 1. *Let $\mathcal{X}$ be an arbitrary set. A kernel $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is positive definite, if, and only if,*

$$\sum_{i=1}^{n} \sum_{j=1}^{n} c_i c_j K(x_i, x_j) \geq 0$$

holds for any $n \in \mathbb{N}$, $\{x_1, \ldots, x_n\} \subseteq \mathcal{X}$ and $\{c_1, \ldots, c_n\} \subseteq \mathbb{R}$.

For a tree kernel, $\mathcal{X}$ is an arbitrary set of trees. When a tree kernel K is positive definite and $\mathcal{X}$ is a finite set, we can view elements (trees) in $\mathcal{X}$ as points in a Euclidean space $\mathcal{H}$, and $K(x, y)$ for $x, y \in \mathcal{X}$ is identical to the inner product of their images in $\mathcal{H}$. This is due to Schönberg's theorem, stated below.

Theorem 1 (Schönberg [2]). *Let $\mathcal{X}$ be an arbitrary set and a kernel $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be positive definite. Then, there exist a Hilbert space $\mathcal{H}$, reproducing kernel Hilbert space, with an inner product operator $\langle \cdot, \cdot \rangle : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ and a mapping $\Phi : \mathcal{X} \to \mathcal{H}$ such that $K(x, y) = \langle \Phi(x), \Phi(y) \rangle$ holds for any $x, y \in \mathcal{X}$.*

A Hilbert space is a linear space equipped with an inner product, which can be of infinite dimension but is complete with respect to the induced norm. For our case with $|\mathcal{X}| < \infty$, $\mathcal{H}$ of Theorem 1 is nothing more than a simple Euclidean space. The most significant advantage of viewing trees as points in Euclidean spaces lies in the fact that we can leverage statistical and machine learning algorithms based on multivariate analysis to analyze tree-structured data. Such algorithms are often referred to as *kernel machines* and include important methods such as support vector machines (SVM), principal component analysis (PCA), k-means clustering, Gaussian-process-based analysis and many others.

The principle of this kernel method is simple. The key component is a *Gramian* matrix: When $\mathcal{X} = \{x_1, \ldots, x_n\}$, the Gramian matrix

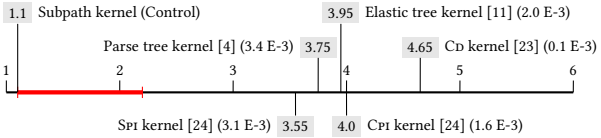

Figure 2: The averaged ranks and the p-values of Hommel test, displayed between parentheses, with the subpath kernel as control. The thick red line represents the critical distance for the significance level 0.01

for $\mathcal{X}$ is the n-dimensional square matrix determined by

$$G(x_1, \ldots, x_n; K) = \begin{bmatrix} K(x_1, x_1) & \ldots & K(x_1, x_n) \\ \vdots & \ddots & \vdots \\ K(x_n, x_1) & \ldots & K(x_n, x_n) \end{bmatrix}.$$

Since K is a positive definite kernel[1], we have

$$G(x_1, \ldots, x_n; K) = \begin{bmatrix} \langle \Phi(x_1), \Phi(x_1) \rangle & \ldots & \langle \Phi(x_1), \Phi(x_n) \rangle \\ \vdots & \ddots & \vdots \\ \langle \Phi(x_n), \Phi(x_1) \rangle & \ldots & \langle \Phi(x_n), \Phi(x_n) \rangle \end{bmatrix}.$$

$\mathcal{H}$, $\langle \cdot, \cdot \rangle$ and Φ are the ones determined in Theorem 1. A support vector classifier (SVC), for example, finds a hyperplane in $\mathcal{H}$ that separates the sets $\mathcal{P} = \{\Phi(x_i) \mid x_i$'s label is $+1\}$ and $\mathcal{N} = \{\Phi(x_i) \mid x_i$'s label is $-1\}$. The criterion for the separating hyperplane is to maximize the margin, the distance from the hyperplane to the closest points of $\mathcal{P} \cup \mathcal{N}$. The closest points are referred to as support vectors [6]. The key fact is that the margin can be computed only using $G(x_1, \ldots, x_n; K)$. For another example, PCA finds a straight line in $\mathcal{H}$ that maximizes the variance of the distribution of the orthographical projections of $\Phi(x_i)$'s to the line, and the variance is also computed using only $G(x_1, \ldots, x_n; K)$. It is important to note that the coordinates of $\Phi(x_i)$ are totally unnecessary for the computation, and only inner products $\langle \Phi(x_i), \Phi(x_j) \rangle$ between $\Phi(x_i)$ and $\Phi(x_j)$ are sufficient.

Many positive definite tree kernels have followed the success of the parse tree kernel in the literature. The *elastic tree kernel* [11] is an important example of tree kernels. The elastic tree kernel as well as the parse tree kernel are guaranteed to be positive definite due to Haussler's theorem [9]. Shin and Kuboyama[23] relaxed the conditions of Haussler's theorem for kernels to be positive definite, and have presented a framework to engineer positive definite kernels efficiently. Many tree kernels have been invented within this framework. In [24] and [21], 64 tree kernels were compared with respect to AUC-ROC measurements of ten-fold cross validation of SVC with ten independent datasets. We draw Fig. 2 to summarize the result of the comparison. The values in the hatched rectangles show the averaged rank across the ten datasets, while the figures in parentheses that follow kernel names are p-values of Hommel test [10] as recommendation by [7]. The best was the subpath kernel, which we describe in details in Section 3.2, and its superiority to the other tree kernels is statistically significant: the p-value of the second best kernel, namely parse tree kernel, is merely 3.4×10^{-3}.

[1] K is positive definite, if, and only if, the Gramian matrix $G(x_1, \ldots, x_n; K)$ has no negative eigenvalues [2].

In addition, to confirm the superiority of the subpath kernel in classification accuracy for our research, we run an additional experiment using a dataset with 25 fake DOM trees and as many authentic DOM trees from our dataset. In the experiment, we compare the subpath kernel with 59 tree kernels including the kernels tested in [24] and [21]. Fig. 3 shows the result. The accuracy scores are measured through ten-fold cross validation. Only the subpath kernel marks 1.0 accuracy, while the scores of all the other kernels fall between 0.54 and 0.93. Thus, the subpath kernel outperforms the others even for the dataset of this paper.

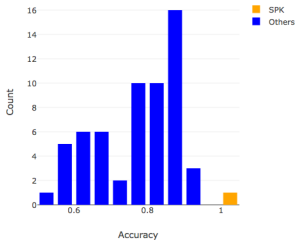

Figure 3: Comparison

3.2 The subpath kernel

From conclusion, we choose the subpath kernel as the standard tree kernel that should be tried first, indeed because of its aforementioned superiority in classification accuracy, and in addition for the following reasons.

- Linear time algorithms to compute the subpath kernel are known [12, 22]. Since the time complexity of computing other tree kernels is mostly of the quadratic order of the size of inputted trees, this advantage is significant. In particular, this enables real-time detection of fake large-scale DOM trees with thousands of vertices.
- The definition of the subpath kernel is invariant no matter whether trees are ordered or unordered: In an ordered tree, orders among sibling vertices are given in addition to their original generation orders. Although DOM trees are derived as ordered trees, adversaries may change sibling orders without impacting their logical meaning or layout. Thus, we see that the results by the subpath kernel is secure against attempts to evade detection by changing sibling order.

3.2.1 The Definition. A subpath π is a contiguous downward path in a tree (Fig. 4). Given two trees x and y, we let $M^{\mathrm{SP}}_{x,y}$ be the set of congruent pair of subpaths. A congruent subpath pair is a subpath in x and another in y of the same length which are also identical as label sequences (Fig. 4). Then, the subpath kernel is defined by

$$K^{\mathrm{SP}}(x, y) = \sum_{(\pi_1, \pi_2) \in M^{\mathrm{SP}}_{x,y}} \lambda^{|\pi_1|}, \quad \lambda > 0.$$

λ is an adjustable hyperparameter referred to as a decay factor (Section 3.2.5), and $|\pi|$ is the length of a subpath π.

3.2.2 Efficient computing of the subpath kernel. In [22], a linear-time algorithm to compute the subpath kernel is presented. It takes a least common prefix (LCP) array as an input. To compute an LCP, we must first extract suffix arrays from trees [22].

3.2.3 Suffix Extraction. For efficiency, the first step of suffix array extraction is assignment of unique identifiers to HTML tags. In this section, we use the following assignment.

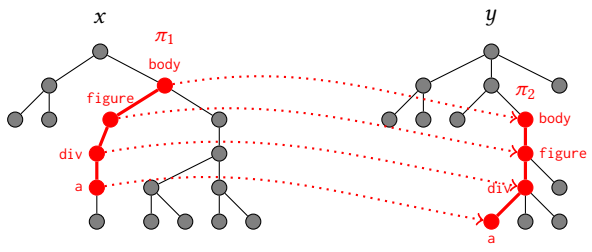

Figure 4: A congruent subpath pair.

Table 1: An LCP array

Suffixes	LCP	$\in x_?$
[3]	1	0
[3]	0	1
[4,3]	0	0
[6,4,3]	0	0
[7,4,3]	0	0
[8,3]	2	0
[8,3]	0	1
[10,24,23,8,3]	1	1
[10,65,8,3]	0	0
[13,17,10,65,8,3]	0	0
[17,10,65,8,3]	0	0
[23,8,3]	3	0
[23,8,3]	0	1
[24,23,8,3]	4	0
[24,23,8,3]	0	1
[25,10,24,23,8,3]	1	1
[25,24,23,8,3]	5	0
[25,24,23,8,3]	0	0
[26,25,10,24,23,8,3]	7	1
[26,25,10,24,23,8,3]	2	1
[26,25,24,23,8,3]	6	0
[26,25,24,23,8,3]	6	0
[26,25,24,23,8,3]	6	0
[26,25,24,23,8,3]	0	0
[65,8,3]	-1	0

3:html 4:head ... 6:title 7:link 8:body
9:js 10:div ... 13:img ... 17:a ...
21:span 22:form 23:table 24:tbody 25:tr
26:td ... 65:figure ...

A suffix is a sequence of the identifiers of HTML tags of a subpath from a vertex of a DOM tree to the root. Fig. 5 exemplifies a DOM tree and all the suffixes extracted from it. Since the DOM tree consists of 17 vertices, 17 suffixes are extracted. For example, a subpath `tr:tbody:table:body:html` corresponds to a suffix `[25,24,23,8,3]`. In the suffix array, suffixes are sorted in lexicographical order. A linear-time algorithm to compute the sorted suffix array is presented in [12].

3.2.4 Least Common Prefix array generation. To generate an LCP array for two trees x and y of 4, the suffix arrays of x and y are first merged, and then, the entries are sorted in the lexicographical order. Table 1 shows an example: suffixes extracted from x are displayed in red and those from y in black, and we assume that x is the tree depicted in Figure 5.

The LCP length for a suffix in the merged array is determined by the length of the longest common prefix between the suffix and the next suffix. For example, the suffixes of `[26,25,10,24,23,8,3]` and `[26,25,24,23,8,3]` are adjacent, and their longest common prefix is `[26, 25]`. Therefore, the LCP length assigned to `[26,25,10,24,23,8,3]` is two as displayed in red. The figure −1 is assigned to the last suffix.

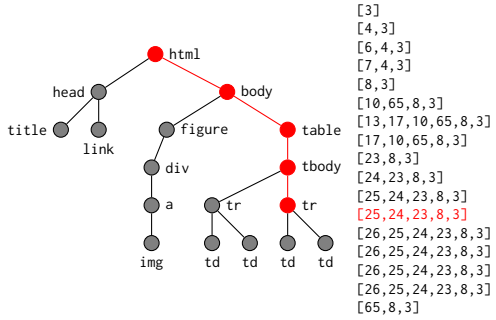

Figure 5: A DOM tree and the associated suffix array.

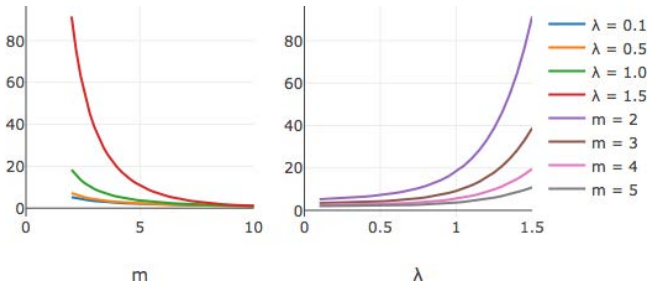

Figure 6: $\gamma(10,\lambda)/\gamma(m,\lambda)$.

3.2.5 Understanding the decay factor λ. When computing $K^{\mathrm{SP}}(x,y)$, a subpath with n vertices shared between x and y is counted by a factor of λ^n. Therefore, the contribution of long subpaths increases, as λ grows. To be precise, if a subpath with n vertices is shared by x and y, they also share i subpaths with $n+1-i$ vertices, and therefore, its contribution to the kernel value is evaluated by

$$\gamma(n,\lambda) = \sum_{i=1}^{n} i\lambda^{n+1-i} = \begin{cases} \frac{\lambda}{\lambda-1}\left(\frac{\lambda^{n+1}-1}{\lambda-1} - n - 1\right), & \text{if } \lambda \neq 1; \\ \frac{n(n+1)}{2}, & \text{if } \lambda = 1. \end{cases}$$

Fig. 6 shows the change of the ratio $\frac{\gamma(10,\lambda)}{\gamma(m,\lambda)}$, when m and λ change. A single subpath with ten vertices is equivalent to 91 subpaths with two vertices for $\lambda = 1.5$, and the number rapidly decreases as m increases and λ decreases.

4 PROPOSED METHOD

In this section, we describe our proposed method step-by-step, and show experimental results. All experiments were conducted on an iMac Pro with 14 Core 2.5 GHz Intel Xeon W 128 Gbyte memory.

4.1 Method overview

Fig. 7 depicts the entire structure of our proposed system. Any HTML documents are first inputted into an HTML parsing/sanitization program, which extracts DOM trees from them after eliminating and correcting non-standardized descriptions included. The obtained DOM trees are inputted into a suffix-extracting program

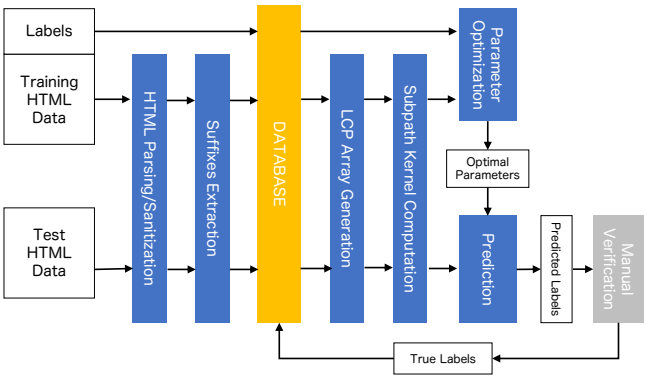

Figure 7: The overall process flow

(Section 3.2.3), and the obtained suffix arrays are stored in a database. Since the suffix array of a DOM tree will be used multiple times to compute the subpath kernel values with other DOM trees, storing the suffix array in a database improves the efficiency of the entire process.

In the next step, the necessary subpath kernel values are computed. When a training dataset consists of DOM trees $x_1, \ldots, x_n$, the complete Gram matrix

$$G(x_1, \ldots x_n) = \begin{bmatrix} k(x_1, x_1) & \ldots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_n, x_1) & \ldots & k(x_n, x_n) \end{bmatrix}$$

is computed. $k(x_i, x_j)$ is the normalized value of the subpath kernel value K^{SP} determined by

$$k(x_i, x_j) = \frac{K^{\mathrm{SP}}(x_i, x_j)}{\sqrt{K^{\mathrm{SP}}(x_i, x_i)}\sqrt{K^{\mathrm{SP}}(x_j, x_j)}}.$$

The normalization is necessary to eliminate undesirable effects caused by different sizes of DOM trees. For prediction, kernel values are computed only between the DOM trees in a test dataset and the DOM trees that comprise support vectors [6] in the training dataset. To compute the subpath kernel value $K^{\mathrm{SP}}(x, y)$, the LCP array between x and y is first computed (Section 3.2.4), and then, is inputted into the linear-time algorithms presented in [22].

During the parameter optimization step, optimal values for the two hyperparameters of the decay factor λ and the regulation coefficient C of SVC are computed through a grid search algorithm.

In the prediction step, a separating hyper-plane in the reproducing kernel Hilbert space is computed with the training dataset and the associated optimal hyperparameters, and then, a predicted label of a DOM tree in the test dataset is computed by SVC.

4.2 Suffix extraction

The suffix array extraction program of our system takes a batch of DOM trees as inputs, and then computes the associated suffix arrays efficiently through parallel computation. Fig. 8 shows the average time in milliseconds to extract suffix arrays from a single DOM tree, when changing the batch size from 50 to 1,000. Although the time scores for $n = 50$ and 100 are higher because of the overhead of parallel computation, we see that our program can process a single DOM tree in 20 milliseconds on average.

Figure 8: Time for suffix array extraction per DOM tree.

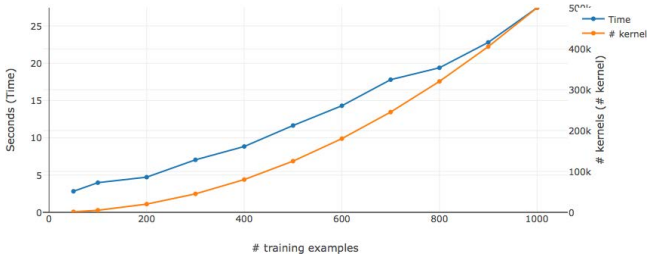

Figure 9: Time for Gram matrix generation.

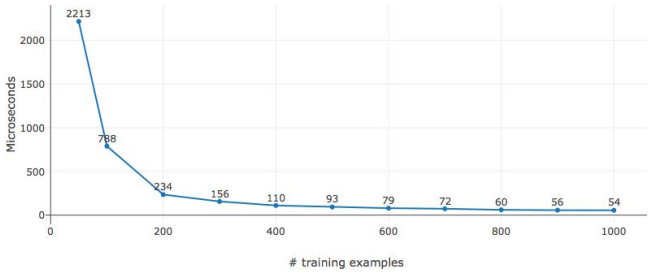

Figure 10: Time for kernel computation per kernel value.

4.3 Gram matrix generation

Since a Gram matrix is symmetric, to compute an n-dimensional Gram matrix, it is necessary to calculate $\frac{n(n+1)}{2}$ kernel values. Hence, the effect of deploying parallel computation is expected to be greater than used for suffix array extraction, because the size of the suffix array generated is proportional to n.

Fig. 9 shows the total run-time scores of computing Gram matrices for n that varies from 50 to 1,000. Although the total run-time (the blue line) should in theory be proportional to the number of kernel values to compute (the orange line), we can see a gap between them that is caused by the overhead of parallel computation.

In fact, Fig. 10 plots the average computation time to compute a single kernel value, or the ratio of the total run-time to the number of kernel values. The effect of the overhead is evident, when n is small. As n increases, the contribution of parallel computation becomes remarkable, and the average time reaches a maximum of 60 microseconds.

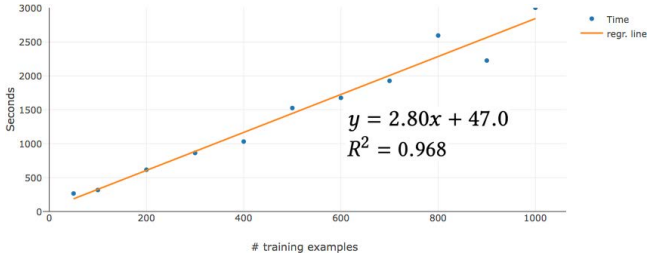

Figure 11: Time for hyperparameter optimization.

4.4 Hyperparameter optimization

To build optimal models, there are two adjustable hyperparameters: the decay factor λ for the subpath kernel and the regulation coefficient C for SVC. Our program for hyperparameter optimization performs a simple grid search by changing λ from 0.1 to 2.0 with an interval 0.1 and $\log_{10} C$ from -5 to 5 with an interval 1. The total number of different combinations of λ and C to test is 220. For evaluation of each combination of λ and C, cross-validation scores with five folds are used. We evaluate the performance of this step from both efficiency and accuracy points of view.

Fig. 11 shows that the relation between the total runtime for optimizing the hyperparameters and the size n of training datasets can be fit to an increasing regression line with a determination coefficient as high as 0.968. Furthermore, the runtime for $n = 1000$ is 3,004 seconds $\approx$ 50 minutes. Due to the excellent generalization performance of SVC, we can obtain good models even for $n \leq 1000$, and hence, models can be updated quickly.

Fig. 12 (a) consists of a plane view and a contour plot of the cross-validation scores (accuracy scores) obtained through the grid search for $n = 1000$. Except for the steep drop-off where the score falls from 1.0 to 0.857 within the rectangular area with $1.4 \leq \lambda \leq 12.0$ and $-5 \leq \log_{10} C \leq -1$, the score is mostly 1.0.

When investigating the cases of fake and authentic sites separately (Fig. 12 (b) and (c)), the accuracy score for the fake sites, computed by $\frac{TP}{TP+FN}$, varies within a narrow range between 0.976 and 1.0, while the score for the authentic sites, computed by $\frac{TN}{TN+FP}$, has a steep drop-off in the same rectangular area as the score for the entire dataset.

For the smaller training datasets we have tested, we observe the same property for most of cases. In particular, the maximum cross-validation is 1.0 for all the cases including when the dataset size is as small as 50.

4.5 Prediction

Next, we test only the parameter combinations that have shown the highest cross-validation score of 1.0 using the 5,946 validation examples. Fig. 13 (a) shows the obtained accuracy scores, when the number of training instances is 1,000. Interestingly, it turns out that the score drops rapidly from 0.98 to 0.8, when the decay factor exceeds 1.3. This implies that overfitting has occurred. When we focus on the fake sites, as Fig. 13 (b) shows, the range of the accuracy score is very narrow between 0.999 and 0.992, and the identification problem of fake sites turns out not to be sensitive to hyperparameter selection. On the other hand, Fig. 13 (c) shows that learning of authentic sites with a decay factor higher than 1.3 is

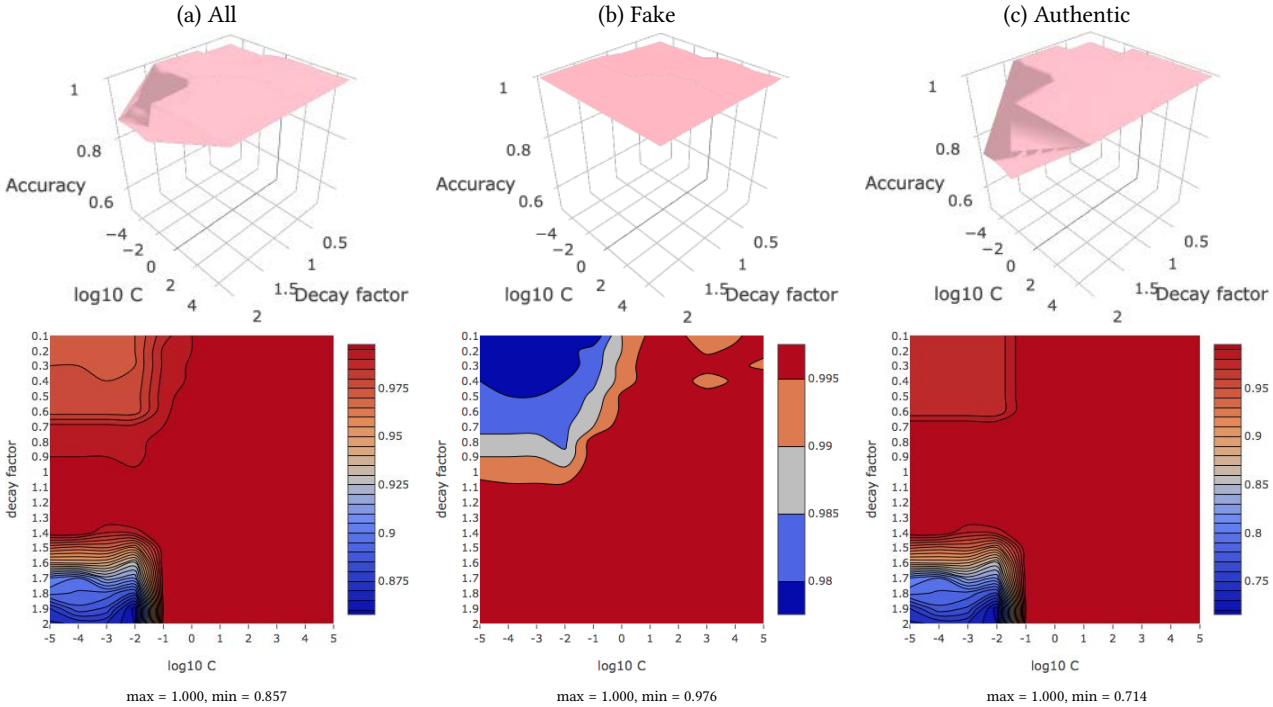

Figure 12: Five-fold cross validation scores for the training dataset of size 1000. In a wide area, the score is as high as 1.0.

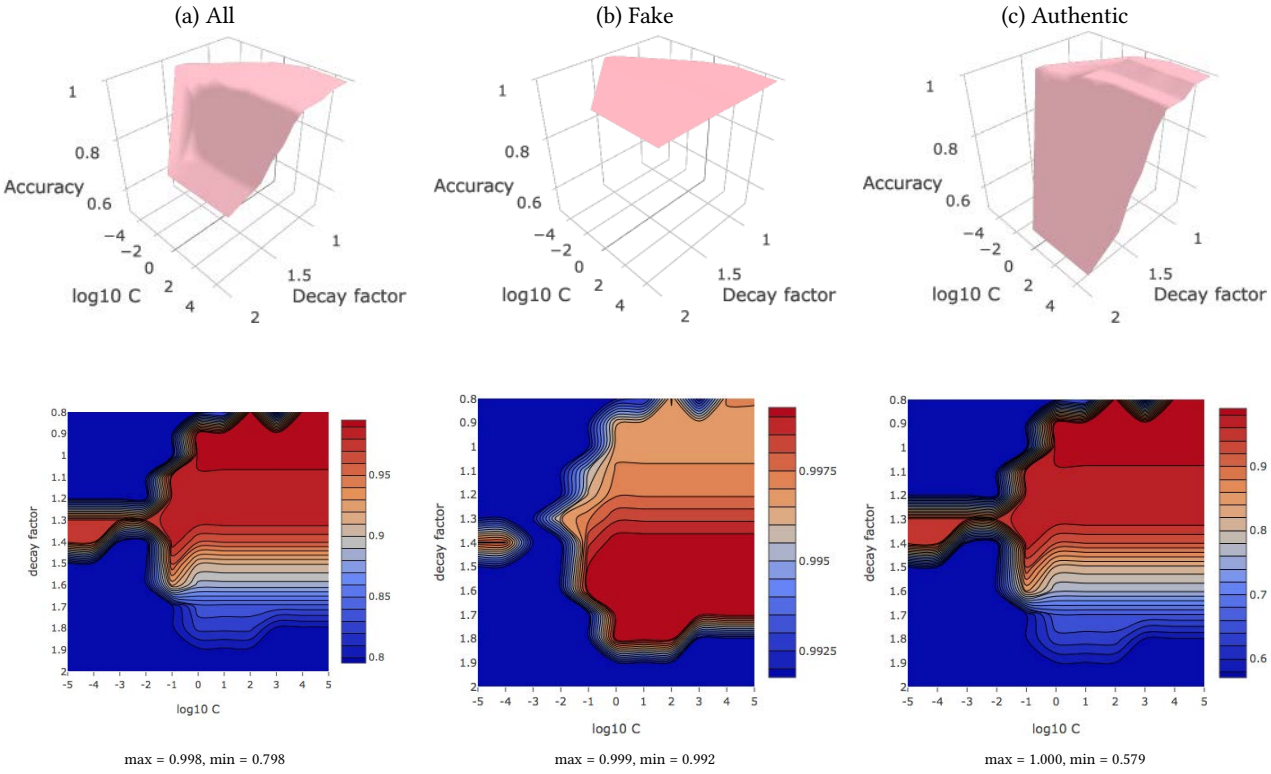

Figure 13: Accuracy scores for 5,946 validation examples with a model that has learnt 1,000 training examples.

likely to generate overfitting models, whose accuracy score can be as low as 0.579.

As seen in Section 3.2.5, the subpath kernel with a high decay factor evaluates longer shared subpaths more significantly. Hence, the results of the validation shows that pages of authentic sites are better characterized by shorter subpaths rather than longer ones.

Furthermore, Fig. 14 shows the results of validation when we use training datasets with fewer than 1,000 examples. Surprisingly, even small datasets can exhibit very high accuracy, and for example, a dataset with only 50 examples shows the accuracy score of 0.994. Also, we can observe overfitting occurring in the same way and under almost the same conditions as when we use a dataset of size 1,000, and SVC shows the best accuracy performance when the parameters are selected from the area of $0.9 \leq \lambda \leq 1.1$ and $0 \leq \log_{10} C \leq 5$.

The first step of prediction is to train an SVC based on the hyper-parameter values obtained in the previous optimization process and the training dataset used. The output of SVC training is a model that specifies the separating hyperplane computed through training.

In our system, we assume that once an SVC is trained, we continue to use the same SVC to make many predictions. Hence, the time efficiency of training an SVC is not significantly critical.

Fig. 15 shows the runtime t in seconds to train an SVC when the size n of a dataset varies. When excluding one outlier ($n = 400$),

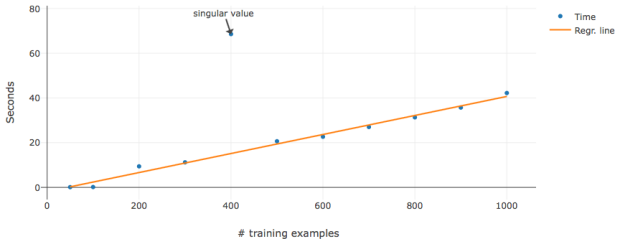

Figure 15: Time for training an SVC.

the relation between t and n fits to a line with determination coefficient 0.990. When $n = 400$, the objective function has converged exceptionally slowly, and the program of libSVM [3] has reached the default upper limit of 10,000,000 iterations. Including this exceptional datum, the total runtime for training does not exceed one minute largely.

The actual output of training an SVC is a subset of examples of a training dataset, which uniquely determines a hyperplane. To make a prediction on a new DOM tree, it suffices to compute the kernel values between the new DOM tree and the support vectors. Since these DOM trees have been converted into suffix arrays in the suffix arrays extraction process, computing these kernel values can

be performed very quickly by taking advantages of the algorithm introduced by [22].

In fact, Fig. 16 shows the averaged runtime to make a prediction for a single unknown DOM tree and the number of support vectors

Figure 14: Accuracy scores for 5,946 validation examples with models that have learnt n training examples: n = 50, 100, 200, 300, 400, 500, 600, 700, 800, 900.

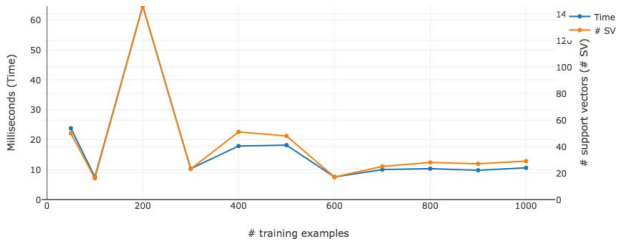

Figure 16: Prediction time and number of support vectors.

for eleven training datasets with different sizes. We can see that the runtime and the number of support vectors are proportional. Although the runtime for $n = 200$ is exceptionally high, it is only 65 milliseconds. For the other datasets the scores are lower than 25 milliseconds. This indicates that our system can perform detection of fake sites in real time.

5 CONCLUSIONS

We have shown that using tree kernels with a support vector classifier allows highly accurate, real-time detection of fake e-commerce sites based on DOM trees. To fulfill our goal, however, choosing an appropriate tree kernel was critical: we tested 60 different kernels with accuracy scores varying from 0.54 to 1.0 before arriving at the subpath kernel. In addition to the advantage in accuracy, the subpath kernel provides extremely high time efficiency. In fact, a linear time algorithm to compute it was presented in the literature very recently, while the time complexity of most of tree kernels is a quadratic function of the size of input trees. Furthermore, the subpath kernel is not affected by vertex sibling order, and hence, a detection mechanism that uses the subpath kernel as a similarity measure is not vulnerable to such attacks as changing tag traversal order.

Additionally, we believe our approach has other features that make it desirable for real-world applications. The accuracy score measured with the validation data was 0.998, and the averaged runtime to predict whether a single DOM tree is fake or authentic is less than 20 milliseconds. Furthermore, it required little training data. Even when we trained with only 50 instances, the accuracy observed was as high as 0.994. This is mainly due to the excellent performance of SVC and the appropriate choice of the subpath kernel. Use of small training datasets yields practical benefit: more efficient and more frequent update of models will be possible.

Lastly, we emphasize that detecting fake sites is only one use of our method. Tree analysis, and therefore subpath kernels, have a broad range of applications. For example, web navigation histories have tree structures, so tree kernels can be used to classify or identify users through web navigation histories. In fact, other research has shown that SVC with tree kernels can distinguish between users from .edu domains and those from other domains. As we saw, since the subpath kernel significantly outperforms other tree kernels both in prediction accuracy and time efficiency, our proposed subpath-kernel based method can apply to a variety of problems in information security, and perform better than existing methods.

REFERENCES

[1] Abbasi, A., and Chen, H. A comparison of tools for detecting fake websites. *Computer* (June 2009).

[2] Berg, C., Christensen, J. P. R., and Ressel, R. *Harmonic Analysis on semigroups. Theory of positive definite and related functions.* Springer, 1984.

[3] Chang, C.-C., and Lin, C.-J. Libsvm: a library for support vector machines. http://www.csie.ntu.edu.tw/˜cjlin/libsvm/, 2001.

[4] Collins, M., and Duffy, N. Convolution kernels for natural language. In *Advances in Neural Information Processing Systems 14 [Neural Information Processing Systems: Natural and Synthetic, NIPS 2001]* (2001), MIT Press, pp. 625–632.

[5] Corona, I., Biggio, B., Contini, M., Piras, L., Corda, R., Mereu, M., Mureddu, G., Ariu, D., and Roli, F. Deltaphish: Detecting phishing webpages in compromised websites. In *ESORICS (1)* (2017), vol. 10492 of *Lecture Notes in Computer Science*, Springer, pp. 370–388.

[6] Cristianini, N., and Shawe-Taylor, J. *An introduction to support vector machines and other kernel-based learning methods.* Cambridge University Press, 2000.

[7] Demšar, J. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Theory 7* (2006), 1 – 30.

[8] Gerbet, T., Kumar, A., and Lauradoux, C. (un)safe browsing. Tech. Rep. RR-8594, INRIA, 2014.

[9] Haussler, D. Convolution kernels on discrete structures. UCSC-CRL 99-10, Dept. of Computer Science, University of California at Santa Cruz, 1999.

[10] Hommel, G. A stagewise rejective multiple test procedure based on a modified bonferroni tests. *Biometrika 75* (1988), 383 – 386.

[11] Kashima, H., and Koyanagi, T. Kernels for semi-structured data. In *the 9th International Conference on Machine Learning (ICML 2002)* (2002), pp. 291–298.

[12] Kimura, D., and Kashima, H. Fast computation of subpath kernel for trees. In *ICML* (2012).

[13] Levenshtein, V. I. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady 10*, 8 (February 1966), 707 – 710.

[14] Li, L., and Helenius, M. Usability evaluation of anti-phishing toolbars. *J. Computer Virology 3* (2007), 163–184.

[15] Liu, W. An antiphishing strategy based on visual similarity assessment. In *IEEE Internet Computing* (2006), pp. 58–65.

[16] Lu, S. Y. A tree-to-tree distance and its application to cluster analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence (PAMI) 1* (1979), 219–224.

[17] Marchal, S., and Asokan, N. On designing and evaluating phishing webpage detection techniques for the real world. In *11th USENIX Workshop on Cyber Security Experimentation and Test (CSET 18)* (Baltimore, MD, 2018), USENIX Association.

[18] Marchal, S., François, J., State, R., and Engel, T. Phishstorm: Detecting phishing with streaming analytics. *IEEE Trans. Network and Service Management 11*, 4 (2014), 458–471.

[19] Marchal, S., Saari, K., Singh, N., and Asokan, N. Know your phish: Novel techniques for detecting phishing sites and their targets. In *ICDCS* (2016), IEEE Computer Society, pp. 323–333.

[20] Satish.S, and Babu.K, S. Phishing websites detection based on web source code and url in the webpage. *International Journal of Computer Science and Engineering Communications 1* (2013).

[21] Shin, K. A theory of subtree matching and tree kernels based on the edit distance concept. *Annals of Mathematics and Artificial Intelligence* (2015).

[22] Shin, K., and Ishikawa, T. Linear-time algorithms for the subpath kernel. In *Proceedings of 29th Annual Symposium on Combinatorial Pattern Matching (CPM 2018)* (2018), pp. 22:1–22:13.

[23] Shin, K., and Kuboyama, T. A generalization of Haussler's convolution kernel - mapping kernel. In *ICML 2008* (2008).

[24] Shin, K., and Kuboyama, T. A comprehensive study of tree kernels. In *JSAI-isAI Post-Workshop Proceedings, Lecture Notes in Articial Intelligence 8417* (2014), Springer, pp. 329–343.

[25] Taï, K. C. The tree-to-tree correction problem. *journal of the ACM 26*, 3 (July 1979), 422–433.

[26] Whittaker, C., Ryner, B., and Nazif, M. Large-scale automatic classification of phishing pages. In *NDSS '10* (2010).

[27] Zhang, K. Algorithms for the constrained editing distance between ordered labeled trees and related problems. *Pattern Recognition 28*, 3 (March 1995), 463–474.

[28] Zhang, K., Wang, J. T. L., and Shasha, D. On the editing distance between undirected acyclic graphs. *International Journal of Foundations of Computer Science 7*, 1 (1996), 43–58.

[29] Zhang, Y., Egelman, S., Cranor, L., and Hong, J. Phinding phish: Evaluating anti-phishing tools. In *Proceedings of 14th Anual Network and Distributed System Security Symposium* (2007), Internet Society.

[30] Zhang, Y., Hong, J. I., and Cranor, L. F. Cantina: A content-based approach to detecting phishing web sites. In *Proceedings of the 16th International Conference on World Wide Web* (New York, NY, USA, 2007), WWW '07, ACM, pp. 639–648.